

Continuous integration using Jenkins

Objective: Required to install Jenkins which can pull code from Github and build project using Maven.

Prerequisites: Install Jenkins, Github, Docker Desktop, DockerHub account (to push docker images), ngrok account(to expose public url of application running on port 8080 i.e, Jenkins so that it can be accessible by github to trigger webhooks). Also install git & github integration plugin in jenkins [So that when code is pushed in git it can trigger Jenkins to do its function]

Step 1: Create directory “myapp”, create **application file** with main code [src/main as shown in github], create a maven structure as a result **pom file** gets created now edit the false pom file(content in screenshots).

- **Pom.xml** file tells how the maven shld build the artifact of the application code (dependencies to include, plugins required).
- During mvn clean package it used pom.xml file along with application code to build artifact and creates target file which contains the artifact[Compiled code, basic tests run, dependencies installed, packaged into jar file]

```
pavan@DESKTOP-4R1FS67: /mnt/c/Users/Pavan/my-app/myapp$ ls
Dockerfile Jenkinsfile pom.xml src target test.txt
```

The screenshot shows a GitHub repository interface for 'maven-docker-jenkins'. The repository is public and has 1 branch (main) and 0 tags. The file structure is as follows:

File	Commit Message	Time
src/main	Changes made in application code	1 hour ago
target	initial commit	17 hours ago
Dockerfile	Dockerfile changes	17 hours ago
Jenkinsfile	pipeline file added	1 hour ago
pom.xml	initial commit	17 hours ago
test.txt	trigger webhook	8 minutes ago

The repository also shows a commit history table with the following details:

Commit	Message	Time
f6187ad	trigger webhook	8 minutes ago

The repository has 8 commits. The right sidebar shows the 'About' section with no description, website, or topics provided. It also shows the 'Releases' section with no releases published and a link to 'Create a new release'. The 'Packages' section shows no packages published.

<> Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Files

main

Go to file

src/main

java/com/mycompany/app

App.java

resources

application.properties

target

Dockerfile

Jenkinsfile

pom.xml

test.txt

maven-docker-jenkins / src / main / java / com / mycompany / app / App.java

Yeshaswini69 Changes made in application code 9d5be2a - 1 hour ago

Code Blame 19 lines (15 loc) · 482 Bytes

```
1 package com.mycompany.app;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.web.bind.annotation.*;
6
7 @SpringBootApplication
8 @RestController
9 public class App {
10
11     public static void main(String[] args) {
12         SpringApplication.run(App.class, args);
13     }
14
15     @GetMapping("/")
16     public String hello() {
17         return "Hello Yeshaswini, from MyApp, running in Docker!";
18     }
19 }
```

Activate Windows
Go to Settings to activate Windows

Server.port=8080 tells tomcat server will be open on port 8080 [container port]

<> Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Files

main

Go to file

src/main

java/com/mycompany/app

App.java

resources

application.properties

target

Dockerfile

Jenkinsfile

pom.xml

test.txt

maven-docker-jenkins / src / main / resources / application.properties

Yeshaswini69 application.properties file change d84cd5b · 16 hours ago

Code Blame 1 lines (1 loc) · 17 Bytes

```
1 server.port=8080
```

Activate Windows
Go to Settings to activate Windows

```
pavan@DESKTOP-4R1FS67: /mnt/c/Users/Pavan/my-app/myapp$ cat pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.app</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.1</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

maven-docker-jenkins / Dockerfile

Yeshaswini69 Dockerfile changes

```
Code Blame 14 lines (10 loc) · 244 Bytes
1 # Use OpenJDK base image
2 FROM eclipse-temurin:17-jdk
3
4 # Set working directory
5 WORKDIR /app
6
7 # Copy the built jar
8 COPY target/myapp-1.0-SNAPSHOT.jar app.jar
9
10 # Expose the app port
11 EXPOSE 8080
12
13 # Run the app
14 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 2: Jenkinsfile . Github creds [to login to github and use the files to run application] and docker hub credentials[to login to github to push the created docker image] must be specified

in Jenkins global credentials and that specific id will be used in Jenkins code as github-creds & docker-creds. Pahe69 is dockerhub username.

- Login to Github using credentials specified, use files from main branch
- Maven build -> use application code files with other config files along with pom file to build artifact of application [compile, tests, dependencies,] and a jar file will be created in the target file in github.
- Docker image -> Use dockerfile[base image, set current working directory, copy target files, expose port, run commands for app to run] to build the docker image, inside the dockerfile the target artifact so created will be used.
- Push Image -> Login to dockerhub using docker creds and push the image
- Run container -> delete existing containers and create a new one to run the application. Port mappings 8081:8080.

```
1  pipeline {
2      agent any
3
4      tools {
5          maven 'Maven3'
6      }
7
8      environment {
9          IMAGE_NAME = "pahe69/myapp"
10         CONTAINER_NAME = "myapp-container"
11     }
12
13     stages {
14
15         stage('Checkout') {
16             steps {
17                 git branch: 'main',
18                     url: 'https://github.com/YesHaswini69/maven-docker-jenkins.git',
19                     credentialsId: 'github-creds'
20             }
21         }
22
23         stage('Build') {
24             steps {
25                 sh 'mvn clean package'
26             }
27         }
28     }
29 }
```

Activate
Go to Setti

```

29     stage('Build Docker Image') {
30         steps {
31             sh 'docker build -t $IMAGE_NAME:latest .'
32         }
33     }
34
35     stage('Push to Docker Hub') {
36         steps {
37             withCredentials([usernamePassword(credentialsId: 'docker-creds',
38             usernameVariable: 'USER', passwordVariable: 'PASS')]) {
39                 sh '''
40                     echo $PASS | docker login -u $USER --password-stdin
41                     docker push $IMAGE_NAME:latest
42                     '''
43             }
44         }
45     }
46
47     stage('Deploy') {
48         steps {
49             sh '''
50                 docker rm -f $CONTAINER_NAME || true
51                 docker run -d -p 8081:8080 --name $CONTAINER_NAME $IMAGE_NAME:latest
52                 '''
53         }
54     }

```

Active
Go to S

Step 3: Make sure **ngrok** is running in separate tab. It is used to expose the public url of the Jenkins app running on 8080 to github. Localhost or privateip can be accessed by github.

```
pavan@DESKTOP-4R1FS67:/mnt/c/Users/Pavan/my-app/myapp$ ngrok http 8080
```

```
pavan@DESKTOP-4R1FS67:/mnt/c/Users/Pavan
```

ngrok

LLMs 4x smaller, 2x faster: <https://ngrok.com/blog/quantization>

```

Session Status      online
Account             yesh690cdyhp@gmail.com (Plan: Free)
Version             3.37.3
Region              India (in)
Latency             190ms
Web Interface       http://127.0.0.1:4041
Forwarding           https://makena-nonteachable-melba.ngrok-free.dev -> http://localhost:8080

```

Connections	ttl	opn	rt1	rt5	p50	p90
	3	0	0.00	0.00	30.17	30.42

HTTP Requests

```

13:32:55.263 +5510 POST /github-webhook/ 200 OK
13:27:20.950 +2010 POST /github-webhook/ 200 OK
13:26:52.381 +5210 POST /github-webhook/ 200 OK

```

Step 4: Create github webhook to signal Jenkins on any push events. Copy the public url given by ngrok accordingly and test the connection through redeliver. Note: Make sure ngrok is running for deliveries to give status 200 Ok, else we wont be able to connect to Jenkins n signal it.

UUCMS - ಸಮಗ್ರ ವಿ...

PREPCA

WhatsApp

YouTube

Gmail

Google

Console home | Con...

+1 other color/patt...

Job Listing | iSt

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [documentation](#).

Payload URL *

https://makena-nonteachable-melba.ngrok-free.dev/github-webhook/

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

Enable SSL verification

Disable (not recommended)

Which events would you like to trigger this webhook?

Just the push event.

Send me everything.

Let me select individual events.

Yeshaswini69 / maven-docker-jenkins

Type to search

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Webhooks / Manage webhook

Settings

Recent Deliveries

aeec39ca-2ffc-11f1-8ee6-d1c1e9bf9ba1

push

2026-04-04 13:32:55

9bea17ce-2ff8-11f1-8a87-f6f59ba5f795

ping

redelivery

2026-04-04 13:27:21

9bea17ce-2ff8-11f1-8a87-f6f59ba5f795

ping

redelivery

2026-04-04 13:26:53

9bea17ce-2ff8-11f1-8a87-f6f59ba5f795

ping

redelivery

2026-04-04 13:21:45

9bea17ce-2ff8-11f1-8a87-f6f59ba5f795

ping

redelivery

2026-04-04 13:09:14

9bea17ce-2ff8-11f1-8a87-f6f59ba5f795

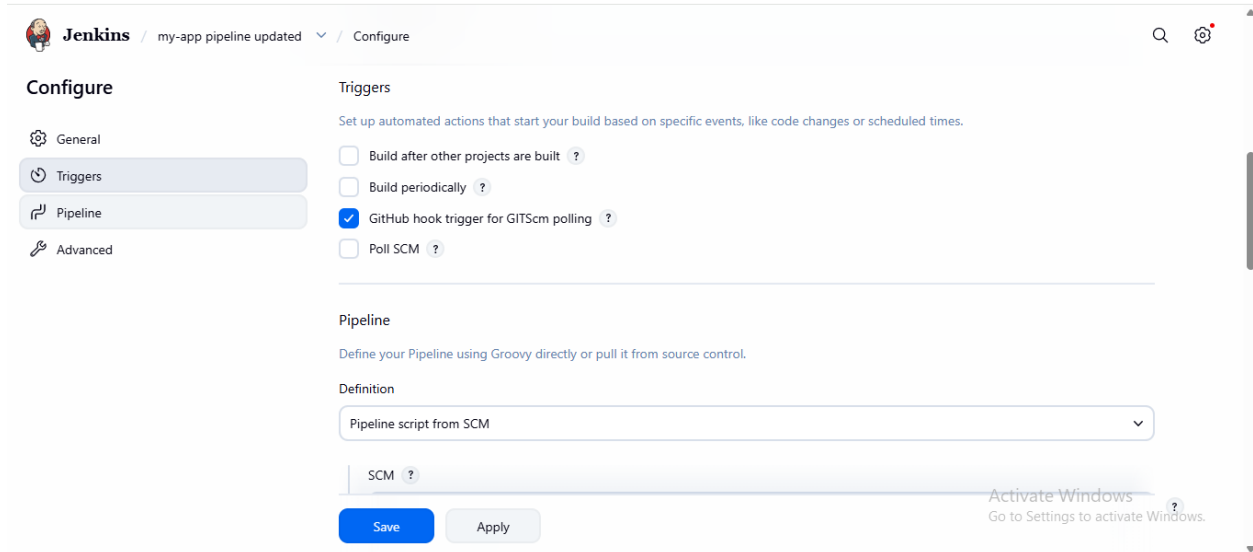
ping

2026-04-04 13:03:45

Activate Windows

Go to Settings to activate Win

Step 5: Configure Jenkins pipeline . Build triggers through webhook GitSCM polling[When code is pushed in github it shld trigger jenkins]. Pipeline script to be used from github with path Jenkinsfile. So when any git push is made Jenkins gets triggered and follows the script from jenkinsfile.



The screenshot shows the Jenkins configuration page for a pipeline named "my-app pipeline updated". The "Configure" tab is selected, and the "Triggers" sub-tab is active. The left sidebar shows the configuration hierarchy: General, Triggers, Pipeline, and Advanced. The "Triggers" section contains three checkboxes: "Build after other projects are built" (unchecked), "Build periodically" (unchecked), and "GitHub hook trigger for GITScm polling" (checked). Below this, the "Pipeline" section is visible, showing the "Definition" dropdown set to "Pipeline script from SCM". The "SCM" dropdown is also set to "Git". At the bottom, there are "Save" and "Apply" buttons. An "Activate Windows" watermark is visible in the bottom right corner.

Jenkins / my-app pipeline updated / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

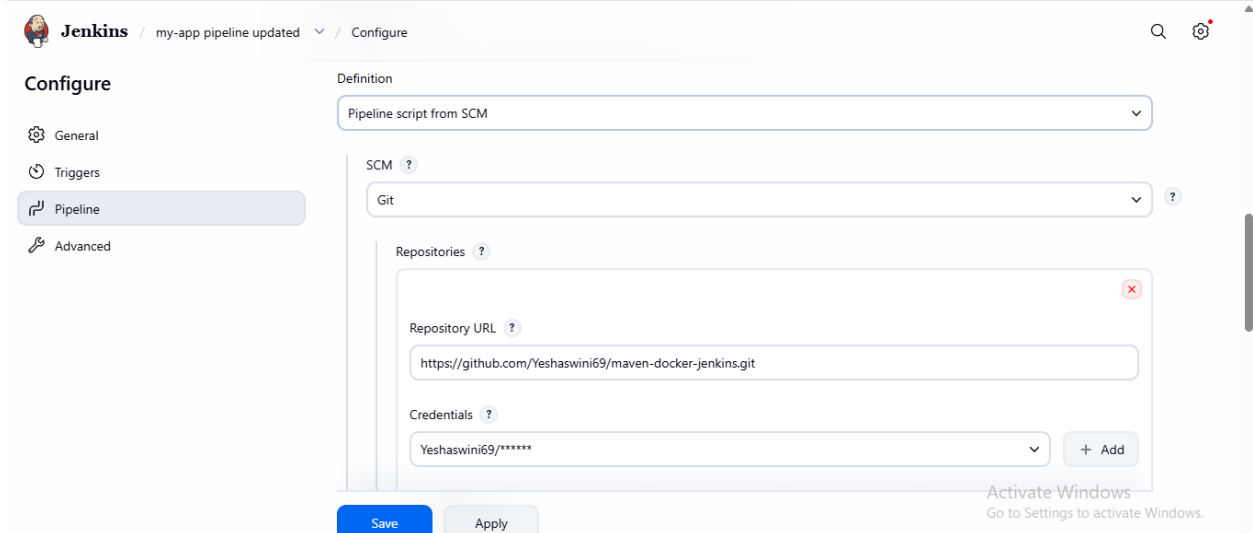
Definition

Pipeline script from SCM

SCM ?

Save Apply

Activate Windows
Go to Settings to activate Windows.



The screenshot shows the Jenkins configuration page for a pipeline named "my-app pipeline updated". The "Configure" tab is selected, and the "Pipeline" sub-tab is active. The left sidebar shows the configuration hierarchy: General, Triggers, Pipeline, and Advanced. The "Pipeline" section contains a "Definition" dropdown set to "Pipeline script from SCM". Below this, the "SCM" dropdown is set to "Git". The "Repositories" section is expanded, showing a "Repository URL" field with the value "https://github.com/Yeshaswini69/maven-docker-jenkins.git" and a "Credentials" dropdown set to "Yeshaswini69/*****". At the bottom, there are "Save" and "Apply" buttons. An "Activate Windows" watermark is visible in the bottom right corner.

Jenkins / my-app pipeline updated / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Definition

Pipeline script from SCM

SCM ?

Git ?

Repositories ?

Repository URL ?

https://github.com/Yeshaswini69/maven-docker-jenkins.git

Credentials ?

Yeshaswini69/*****

+ Add

Save Apply

Activate Windows
Go to Settings to activate Windows.

Jenkins / my-app pipeline updated / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Branch Specifier (blank for 'any') ?

main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

Save Apply

Activate Windows
Go to Settings to activate Windows.

Step 6: Make some changes and push the code to github origin main.

```
pavan@DESKTOP-4R1FS67: /mnt/c/Users/Pavan/my-app/myapp$ echo "trigger webhook" >> test.txt
git add .
git commit -m "trigger webhook"
git push origin main
[main f6187ad] trigger webhook
1 file changed, 1 insertion(+)
create mode 100644 test.txt
Username for 'https://github.com': Yeshaswini69
Password for 'https://Yeshaswini69@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 291 bytes | 13.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Yeshaswini69/maven-docker-jenkins.git
9d5be2a..f6187ad main -> main
```

Step 7: Jenkins build is started automatically. We can check github webhook to see the Jenkins connection status.

Github files used, maven build, docker image build, image push to docker hub [check the docker hub to see the latest image created], container started which runs the application.


```
[Pipeline] {
[Pipeline] sh
+ docker rm -f myapp-container
myapp-container
+ docker run -d -p 8081:8080 --name myapp-container pahe69/myapp:latest
0810cd0a7182a6e527b088bfc7955c3667a1346a51782d0998b37d3045fb19a2
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Activate Window
Go to Settings to activate Windows.

hub

Explore My Hub

Search Docker Hub

CtrlK

P

pahe69

Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Upgrade plan

Repositories

All repositories within the pahe69 namespace.

Search by repository name

All content

Create a repository

Name	Last Pushed	Contains	Visibility	Scout
pahe69/myapp	25 minutes ago	IMAGE	Public	Inactive

1-1 of 1 < >

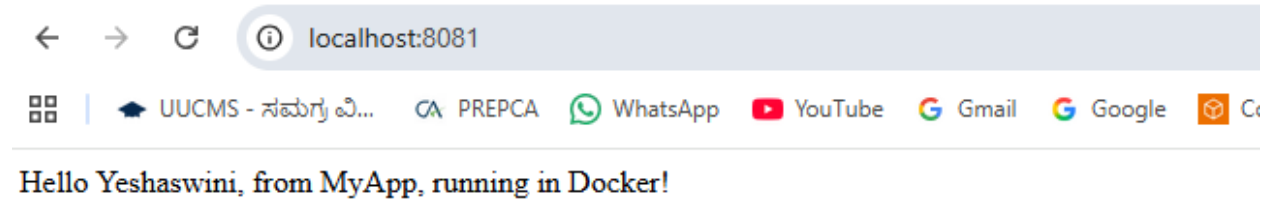
Activate Windows

Go to Settings to activate Windows.

Container created and running

```
pavan@DESKTOP-4R1FS67: /mnt/c/Users/Pavan/my-app/myapp$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0810cd0a7182   pahe69/myapp:latest   "java -jar app.jar"     25 minutes ago   Up 25 minutes   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   myapp-container
pavan@DESKTOP-4R1FS67: /mnt/c/Users/Pavan/my-app/myapp$
```

Step 8: Verify if the application is running on port 8081. So anytime code changes are made and pushed to github, webhook triggers which triggers Jenkins and Jenkins script gets executed where maven build, docker image build , push image to dockerhub and container runs which will have the updated application running. Hence the Continuous integration and deployment.



Pro Tip to remember:

1. GitBash -> Create files [Application file, pom.xml file, Jenkins file]
2. Github -> Push files to GitHub, configure Github webhook
3. Jenkins -> Pipeline configured, install required plugins
4. Commit new changes and git push
5. Check github webhook status, ngrok should be running
Note: Usually Jenkins will be installed on remote server in that cases we don't need to use ngrok cz each time it is started it gives new public url which is not recommended.
6. Pipeline build automatically starts
7. Application live with updated features.